

Restorable Segmentation Synthesis (RSS) Tool User Guide

Contents

Version (last update): 8/20/2025	3
Introduction.....	3
Purpose.....	3
The Use of GUI.....	4
Load a Truth Mask.....	4
Single Synthetic Segmentation Generation.....	5
Parameter Adjustment	6
Single Visualization	6
Segmentation Metrics	7
Multiple Synthetic Segmentations Generation	7
Visualization of Multiple Segmentations.....	8
Metric Statistics	8
Segmentation Fusion.....	9
Selection Mode	11
Batch Processing.....	11
Generation from Multiple Masks	12
User Segmentation Fusion	13
Random Seed	13
The Use of Codes (Core Functions).....	14
Synthetic Segmentation Generation.....	14
Function for converting a binary mask to the Fourier Descriptors (FDs): bd2Fdesc	14
Function for adding Gaussian noise to FDs: FD_change	14
Function for converting the FDs back to a contour: Fdesc2bd	15
Function for filling a contour to a mask: contour2mask	15
Segmentation Fusion.....	15
Function for averaging masks: Fusion_MEAN	16
Function for fusing masks by MV: Fusion_MV	16
Function for fusing masks by TESD: Fusion_TESD	16
Function for fusing masks by STAPLE: Fusion_STAPLE	16

Segmentation Metrics	16
Function for computing Dice Coefficient: DICE	17
Function for computing Jaccard Coefficient: JAC	17
Function for computing Hausdorff Distance: HD	17
Function for computing Medical Similarity Index: MSI	17

Version (last update): 8/20/2025

Introduction

The Restorable Segmentation Synthesis (RSS) Tool is software written in **Python** that allows a user to generate synthetic segmentation contours by synthesizing controlled segmentation errors on pre-defined truth contours. The RSS tool can be used through a graphical user interface (GUI) or as command-line functions inserted into a user's own code. The GUI allows for visualization of the synthesis segmentation, interactive tuning of the synthesis parameters, and display of several segmentation evaluation results. The command-line mode allows for processing images in batches as well as providing flexible ways for users to integrate the RSS tool with their applications.

The RSS tool provides **image restorable segmentation synthesis function**. This function was designed such that the average across the synthetic contours asymptotically converges to the original truth contour. This allows evaluation of truthing methods by simulation of multiple observers' segmentations that can be fused with a truthing method to define a reference standard (truth). More importantly, the RSS tool enables the creation of benchmark datasets to compare different truthing methods and can also be used for data augmentation in training AI for medical imaging.

Purpose

The RSS tool can be used for:

- Investigating properties of segmentation performance metrics and informing segmentation metric selection.
- Investigating truthing methods and informing truthing method selection by allowing users to assess the impact of different truthing methods for combining multiple segmentation (truth) masks provided by a set of truthers.
- Augmenting segmentation contours for generative AI models to generate images for improving training of AI models.

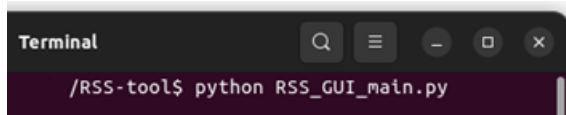
The clinical applications can be AI development/validation in any medical imaging modality such as Digital Pathology and Radiology imaging.

The Use of GUI

For information installation, please refer to the GitHub webpage of RSS tool:

<https://github.com/DIDSR/RSS-tool/>

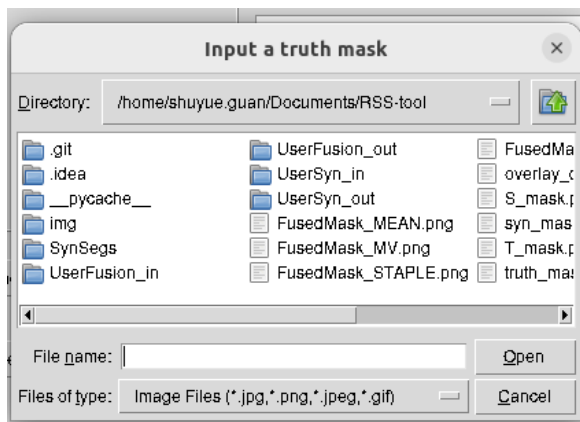
To start the GUI, under the main folder of “RSS-tool”, run “RSS_GUI_main.py” file in Python:



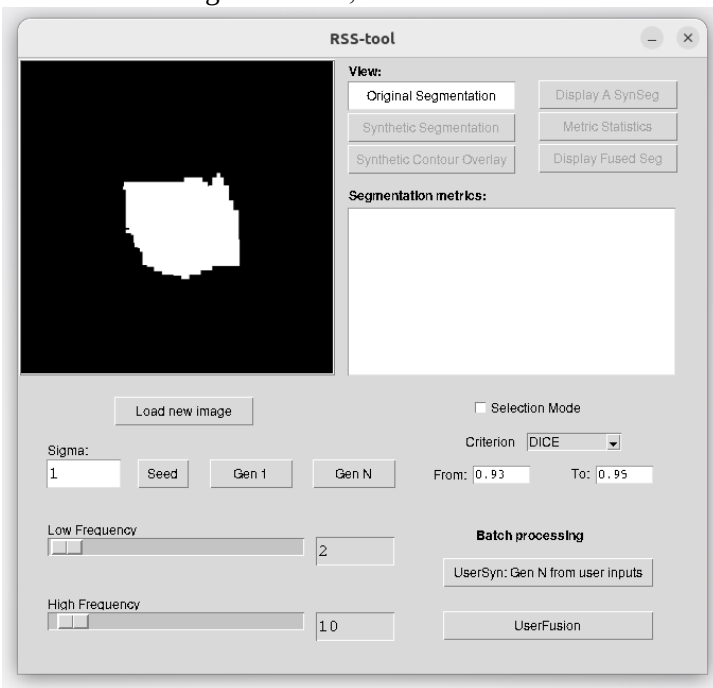
Load a Truth Mask

When running the RSS tool GUI, the first step is to input a truth mask. You can choose a mask image from the pop-up window.

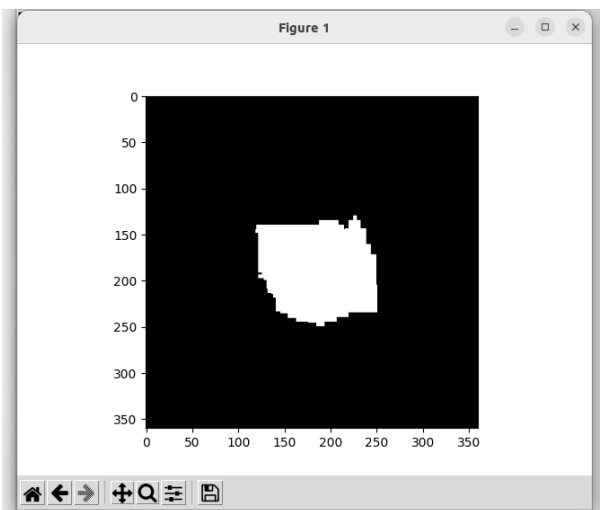
* The input mask must be a binary image.



After a mask image is chosen, the “RSS-tool” main window is shown.



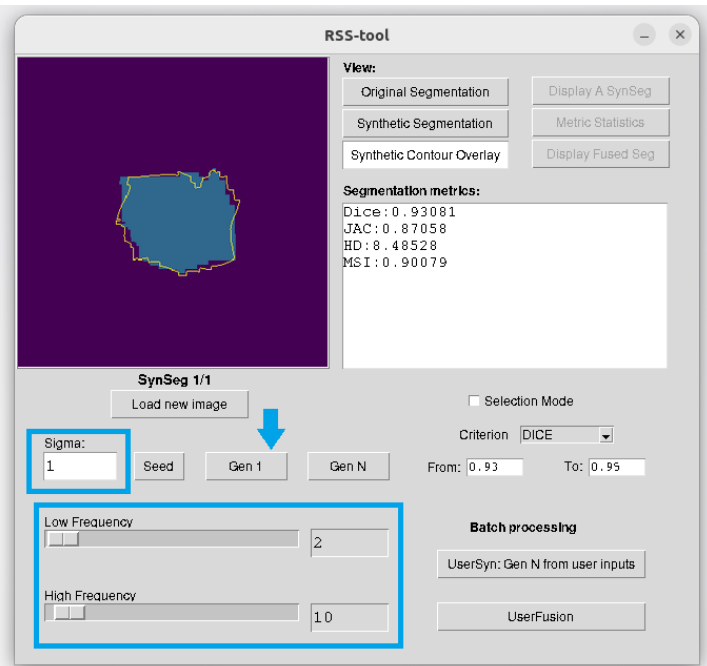
Where the chosen mask is displayed. If you **click the image box**, a pop-up window allows to zoom or save the image.



To change to a different mask image, click “**Load new image**” button.

Single Synthetic Segmentation Generation

To generate a synthetic segmentation from the loaded truth mask, click the “**Gen 1**” button (blue arrow).



* The synthetic segmentation changes randomly every time the “Gen 1” button is clicked.

Parameter Adjustment

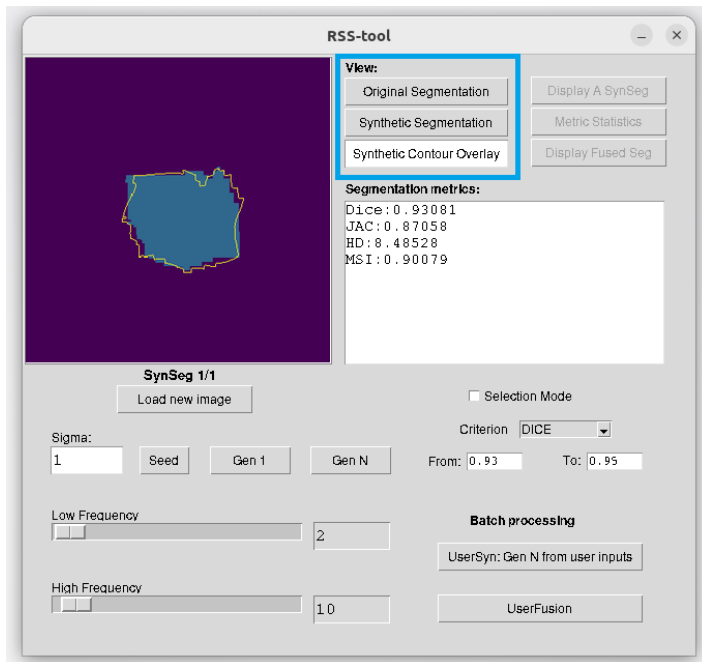
“Gen 1” button: one synthetic segmentation is generated from the truth mask using three parameters: **Sigma**, **low frequency**, and **high frequency** (blue boxes above).

* The high frequency must be greater than the low frequency.

For details of these parameters, please see the manuscript: *(To be published; tentative link for internal review: <https://www.overleaf.com/read/yrxpsvpcvgzs#0c0c90>)*. Briefly, the low frequency components reflect the global shape of the contour and the high frequency ones reflect contour’s local details. The frequency band between the Low Frequency and High Frequency is modified by the tool to generate a new contour.

Single Visualization

Once a synthetic segmentation is generated, the **view option** (blue box below) is available to view the **Original Segmentation**, **Synthetic Segmentation**, or the **Synthetic Contour Overlay**.



Segmentation Metrics

Four segmentation metrics comparing the Original (truth) and Synthetic Segmentations are listed:

- **DICE** Dice Coefficient
- **JAC** Jaccard Coefficient
- **HD** Hausdorff Distance
- **MSI** Medical Similarity Index

References and definitions for metrics: the **MSI** is from Kim, Haksoo, et al. "Quantitative evaluation of image segmentation incorporating medical consideration functions." Medical physics 42.6Part1 (2015): 3013-3023. <https://doi.org/10.1118/1.4921067> , and other metrics are from Taha, A.A., Hanbury, A. Metrics for evaluating 3D medical image segmentation: analysis, selection, and tool. BMC Med Imaging 15, 29 (2015). <https://doi.org/10.1186/s12880-015-0068-x>

Multiple Synthetic Segmentations Generation

To generate multiple synthetic segmentations from the loaded mask using the same set of parameters, click the **“Gen N”** button. Then, it asks the user to enter the number (N) of synthetic segmentations to be generated. The N synthetic contours are saved in the **“SynSegs”** folder, named **“synseg_x_.png”**, where x runs from 1 to N.

After running, **all parameters are saved in the file “GanN_params_YYMMDD-HHMMSS.txt”** in the root folder. *Date and time are recorded in YYMMDD-HHMMSS format.*

* All existing images in **“SynSegs”** folder will be firstly **deleted** after new round of multiple generation. **“Gen 1”** does not affect them. Please move/copy the images if you want to keep them.

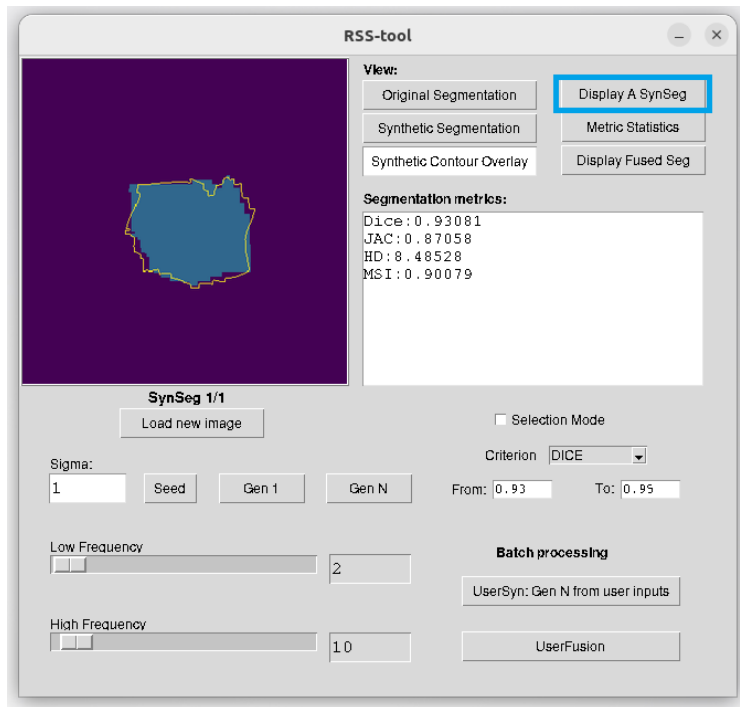
* All functions latter in this section **depend** on the images in **“SynSegs”** folder.

We also provide the “**UserSyn: Gen N from user inputs**” function in Batch Processing to generate groups of synthetic segmentations from multiple masks (M to M×N generation) by one click. Please find the detail in the [Batch Processing](#) section.

Visualization of Multiple Segmentations

To view the results of “Gen N”, you can open them directly in the “**SynSegs**” folder.

To view one generated segmentation overlaid on the original segmentation, click the “**Display A SynSeg**” button and input the index number for a generated segmentation you want to see. For example, “2” is for the “synseg_2_.png”.

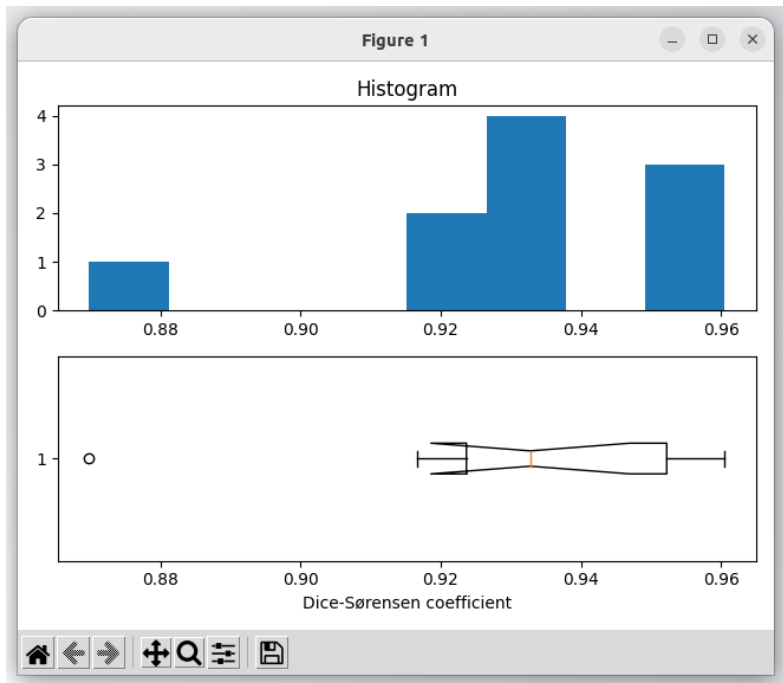


Metric Statistics

We provided the metric statistic function for the N synthetic segmentations. Click the “**Metric Statistics**” button you will see the options of four metrics:

- DICE Dice Coefficient
- JAC Jaccard Coefficient
- HD Hausdorff Distance
- MSI Medical Similarity Index

Each option results in a histogram and a boxplot of values of the selected metric. Here shows an example of 10 synthetic segmentations using DICE:



This function can help verify the results in [Selection Mode](#) – to confirm all synthetic data are in the designated metric range.

Segmentation Fusion

The RSS tool offers several mask-fusion functions, which can be applied to combine synthetic segmentations into a reference standard. Click the “Display Fused Seg” button you will see the options of four fusion functions:

- [Mean](#)
- [Majority Vote \(MV\)](#)
- [Truth Estimate from Self Distances \(TESD\)](#)
- [Simultaneous Truth And Performance Level Estimation \(STAPLE\)](#)

References/definitions:

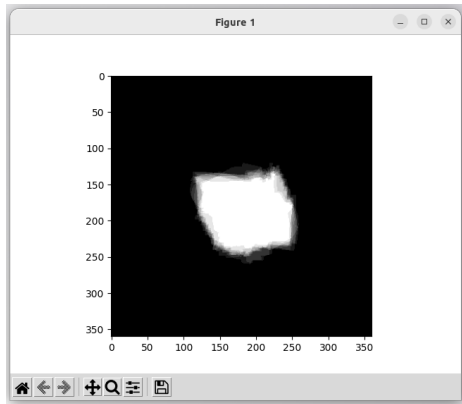
Mean – An averaged mask of the N synthetic segmentations. **Only this result in a gray-scale image, others in a binary mask.*

MV - Menze, B. H. et al., The multimodal brain tumor image segmentation benchmark (brats),” IEEE Transactions on Medical Imaging 34(10), 1993–2024 (2015).

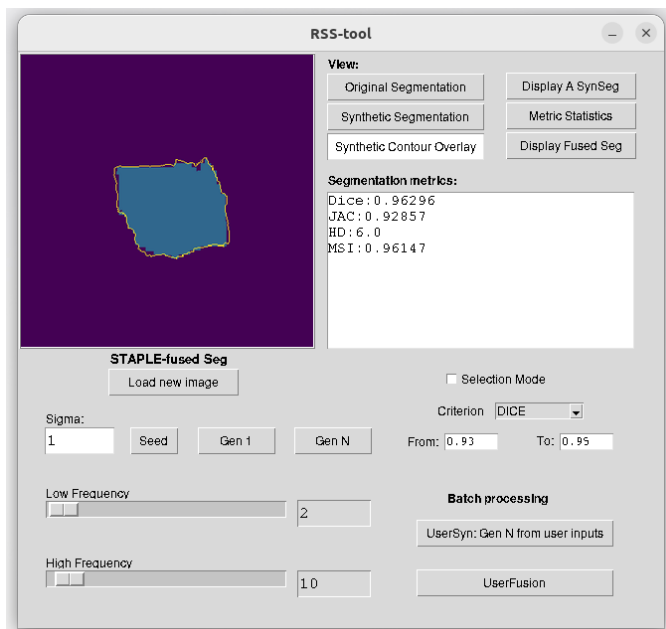
TESD - Biancardi, A. M. and Reeves, A. P., “Tesda: a novel ground truth estimation method,” in [Medical Imaging 2009: Computer-Aided Diagnosis], 7260, 1116–1123, SPIE (2009).

STAPLE - Warfield, S. K., Zou, K. H., and Wells, W. M., “Simultaneous truth and performance level estimation (STAPLE): an algorithm for the validation of image segmentation,” IEEE transactions on medical imaging 23(7), 903–921 (2004).

Fusion results are displayed in different ways: for Mean, it displays in a pop-up figure window. Here shows a fusion result of 10 synthetic segmentations using Mean:



Other fusion results are displayed in the main window **as a binary mask**. You can compare the fusion result with the original segmentation by the overlapped image of them (**Synthetic Contour Overlay**). Segmentation metrics comparing the original and the fusion result are also displayed. Here shows a fusion result of 10 synthetic segmentations using STAPLE:

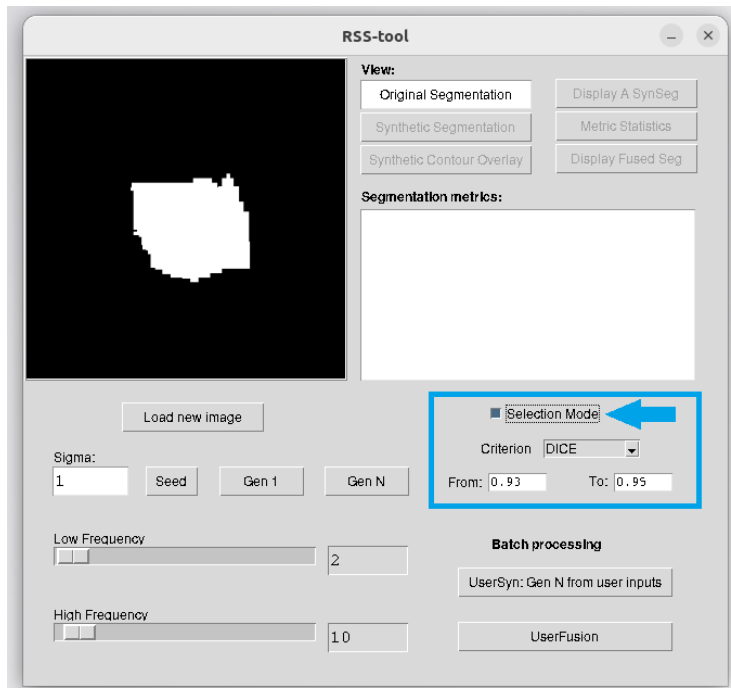


We also provide the “**UserFusion**” function in Batch Processing to fuse many groups of masks to fusion results by one click. Please find the detail in the [Batch Processing section](#).

Selection Mode

Before setting this mode ON, we strongly recommend users to examine the distribution of metrics using the Metric Statistics function and set the parameters accordingly. For any input, the program will stop running after 200 trials if none of the synthetic segmentation can be selected.

Check the “**Selection Model**” to enable the selection of synthetic segmentation during generation according to a user defined criterion (see below).



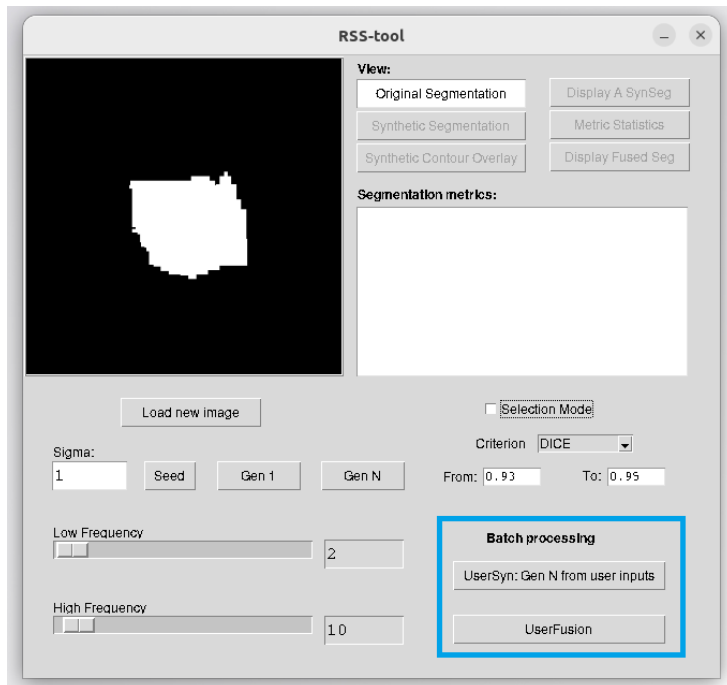
*The Selection Model is effective only for “Gen N” and “UserSyn: Gen N from user inputs” including images output, **NOT applied on the “Gen 1” function**.

When the Selection Model is on, synthetic segmentations are selected only if the values of **Criterion** measuring the similarity of the synthetic segmentation and the original segmentation are within the range as defined by the “**From**” and “**To**” parameters.

For example, as shown in the figure, for all generated synthetic segmentations (by using “Gen N” and “UserSyn: Gen N from user inputs”), their *Dice Coefficient* values are from 0.93 to 0.95.

Batch Processing

RSS tool provides Batch Processing functions for 1) synthetic segmentation generation “**UserSyn**” and 2) Segmentation Fusion “**UserFusion**” (by the selected truthing method). The Batch Processing functions are shown in the blue box.



Here summarizes synthetic segmentation generation and segmentation fusion functions in RSS tool and shows their differences:

- Synthetic Segmentation Generation
 - From 1 (original) to 1 (synthetic): “Gen 1”
 - From 1 to N: “Gen N”
 - From M to M×N: “UserSyn”
- Segmentation Fusion
 - From N to 1: “Display Fused Seg”
 - From M×N to M: “UserFusion”

Generation from Multiple Masks

By clicking the “**UserSyn**” button, users are asked for setting a number N, which is the number of synthetic segmentations will be generated from each original mask in a folder (default: “**UserSyn_in**”) and saved in another folder (default: “**UserSyn_out**”). To customize the input/output folders, please edit `<batch_processing_config.txt>`:

- UserSyn_mask_folder=UserSyn_in/
- UserSyn_output_folder=UserSyn_out/

* Change of folders must be applied before clicking the “**UserSyn**” button.

After running the function, synthetic segmentations can be found in the output folder (default: “**UserSyn_out**”). The names of subfolders are the names of original masks. All synthetic segmentations generated from one original mask are saved in the same subfolder. **And all parameters are saved in the**

file “UserSyn_params_YYMMDD-HHMMSS.txt” in the root folder. Date and time are recorded in YYMMDD-HHMMSS format.

User Segmentation Fusion

By clicking the “**UserFusion**” button, masks in each subfolder under a folder (default: “**UserFusion_in**”) will be fused by the three truthing methods (MV, TESD, and STAPLE; more details in the [Segmentation Fusion section](#)) and saved in another folder (default: “**UserFusion_out**”). Three fused masks from the same subfolder under the input folder are saved in the same subfolder under the output folder. Each fused mask is fused by all masks in a subfolder under the input folder by using one of the three truthing methods.

To customize the input/output folders, please edit <batch_processing_config.txt>:

- UserFusion_folder_in=UserFusion_in/
- UserFusion_folder_out=UserFusion_out/

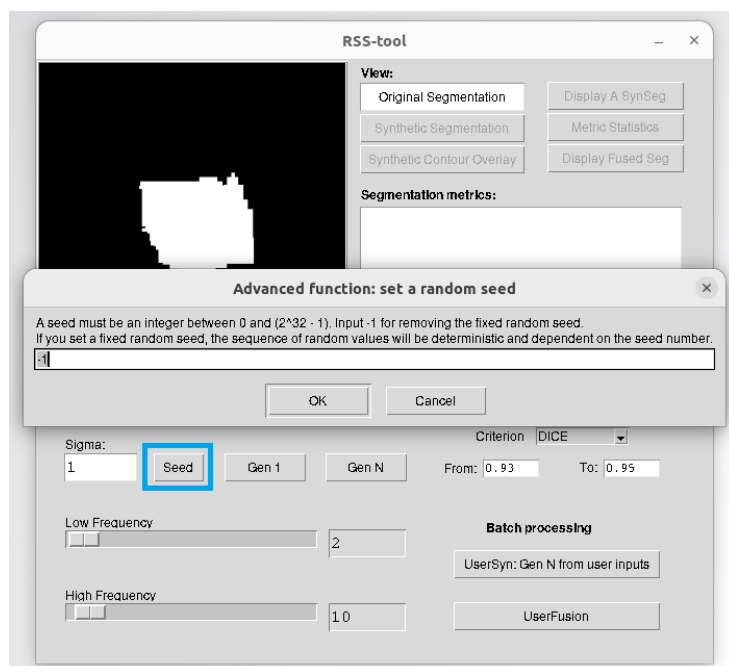
* Change of folders must be applied before clicking the “**UserFusion**” button.

Random Seed

By default, RSS tool uses [numpy.random.normal](#) with [numpy.random.seed](#) (seed=None) to generate noise values added to the segmentation contour. That the Random Seed is set to None means for every time it creates a random value, the pseudo-random number generator (PRNG) will be initialized using a source of entropy from the operating system, typically the current system time. In other words, the Random Seed changes each time.

To set a fixed Random Seed, click the “**Seed**” button (the blue box) and input a non-negative integer.

Note: “-1” means None (to remove the fixed Random Seed) and only integers between 0 and ($2^{32} - 1$) are acceptable as Random Seeds.



* Once set, the fixed Random Seed is applied to **ALL** generation functions in RSS tool (including “Gen 1”, “Gen N”, and “UserSyn”).

* Once set, the **sequence of random values** used by functions in RSS tool will be **RESET**, fixed, and dependent on the seed number.

Reset the sequence of random values:

- Set the Random Seed with the same value again.
- Restart the RSS tool, then set the Random Seed with the same value.

Remove the fixed Random Seed:

- Set the Random Seed = “-1”.
- Restart the RSS tool.

The Use of Codes (Core Functions)

The codes of main functions include three parts:

- Synthetic Segmentation Generation
- Segmentation Fusion
- Segmentation Metrics

Synthetic Segmentation Generation

Four functions are used for the Synthetic Segmentation Generation:

- **bd2Fdesc**: convert a binary mask to the Fourier Descriptors (FDs) describing its contour.
- **FD_change**: add Gaussian noise to FDs.
- **Fdesc2bd**: convert the FDs back to a contour.
- **contour2mask**: fill a contour to a mask and check if it is a non-empty and closed contour with only one connected component.

The four functions are included in the “**IPfunctions.py**” file. They can be imported by: `<from IPfunctions import *>` in Python.

*Function for converting a binary mask to the Fourier Descriptors (FDs): **bd2Fdesc***

Usage: `fd = bd2Fdesc(mask)`

Input: `{mask}`: binary mask image.

Output: A $N \times 2$ matrix. N is the number of FDs to describe contour of the mask. Its first column is real part of FDs, and second column is imagery part of FDs.

*Function for adding Gaussian noise to FDs: **FD_change***

Usage: `fd = FD_change(fd, l, h, sigma)`

Input: *{fd}*: FDs to describe the contour. A $N \times 2$ matrix. Its first column is real part of FDs, and second column is imagery part of FDs.

{l, h}: The range from low (l) to high (h) frequency in FDs to be changes. The high frequency must be greater than the low frequency.

{sigma}: The standard deviation of the Gaussian noise. To change the FDs, the values of `numpy.random.normal(0, sigma)` are added to both real and imagery part of FDs.

More details about the parameters to change the FDs can be found in the previous section: [Parameter Adjustment](#).

Output: Changed FDs to describe the modified contour. A $N \times 2$ matrix. Its first column is real part of FDs, and second column is imagery part of FDs.

Function for converting the FDs back to a contour: Fdesc2bd

Usage: `contour = Fdesc2bd(fd, size)`

Input: *{fd}*: FDs to describe the contour. A $N \times 2$ matrix. Its first column is real part of FDs, and second column is imagery part of FDs.

{size}: The size of the image including the contour. This size should be the same as the original input mask image.

Output: binary contour image.

Function for filling a contour to a mask: contour2mask

Usage: `mask, eligible_flag = contour2mask(contour)`

Input: *{contour}*: binary contour image.

Output: *{mask}*: a useful binary mask image filling by contour, if *eligible_flag* = true.

{eligible_flag}: if the contour is non-empty, closed, and has only one connected component; Boolean.

- *true*: the contour is 1) non-empty, 2) closed, and 3) has only one connected component.
- *false*: at least one of the three requirements is not fulfilled.

Segmentation Fusion

Four functions are used for the Segmentation Fusion:

- **Fusion_MEAN**: average N masks to a gray-scale image.
- **Fusion_MV**: fuse N masks to one by using Majority Vote (MV).
- **Fusion_TESD**: fuse N masks to one by using Truth Estimate from Self Distances (TESD).
- **Fusion_STAPLE**: fuse N masks to one by using Simultaneous Truth And Performance Level Estimation (STAPLE).

The technique details of these fusion methods can be found in the [Segmentation Fusion section](#).

The four functions are included in the “**LabelFusion.py**” file. They can be imported by: <from LabelFusion import *> in Python.

*Function for averaging masks: **Fusion_MEAN***

Usage: **Fusion_MEAN(MaskFolder, output_folder)**

Input: *{MaskFolder}*: the folder only includes binary masks to be averaged.

{output_folder}: the output folder for the averaged gray-scale image.

Output: None return from the function. The fused mask is saved in the output folder.

*Function for fusing masks by MV: **Fusion_MV***

Usage: **Fusion_MV(MaskFolder, output_folder)**

Input: *{MaskFolder}*: the folder only includes binary masks to be fused.

{output_folder}: the output folder for the fused binary image.

Output: None return from the function. The fused mask is saved in the output folder.

*Function for fusing masks by TESD: **Fusion_TESD***

Usage: **Fusion_TESD(MaskFolder, output_folder)**

Input: *{MaskFolder}*: the folder only includes binary masks to be fused.

{output_folder}: the output folder for the fused binary image.

Output: None return from the function. The fused mask is saved in the output folder.

*Function for fusing masks by STAPLE: **Fusion_STAPLE***

Usage: **Fusion_STAPLE(MaskFolder, output_folder)**

Input: *{MaskFolder}*: the folder only includes binary masks to be fused.

{output_folder}: the output folder for the fused binary image.

Output: None return from the function. The fused mask is saved in the output folder.

Segmentation Metrics

RSS tool provides functions to compute four segmentation metrics:

- **DICE:** Dice Coefficient
- **JAC:** Jaccard Coefficient

- **HD:** Hausdorff Distance
- **MSI:** Medical Similarity Index

The technique details of these metrics can be found in the [Segmentation Metrics](#) section.

The four functions are included in the “[seg_measures.py](#)” file. They can be imported by: `<from seg_measures import *>` in Python.

*Function for computing Dice Coefficient: **DICE***

Usage: `dice = DICE(truth_mask, seg)`

Input: `{truth_mask, seg}`: Two binary masks to be compared.

Output: The Dice Coefficient value of the two masks/segmentations.

*Function for computing Jaccard Coefficient: **JAC***

Usage: `jac = JAC(truth_mask, seg)`

Input: `{truth_mask, seg}`: Two binary masks to be compared.

Output: The Jaccard Coefficient value of the two masks/segmentations.

*Function for computing Hausdorff Distance: **HD***

Usage: `hd = HD(truth_mask, seg)`

Input: `{truth_mask, seg}`: Two binary masks to be compared.

Output: The Hausdorff Distance value of the two masks/segmentations.

*Function for computing Medical Similarity Index: **MSI***

Usage: `msi = MSI(ref_img, test_img, il=1, ol=1)`

Input: `{ref_img, test_img}`: Two binary masks to be compared.

`{il, ol}`: The two parameters: il (inside level) and ol (outside level) are defined in MSI. For their details, see the [paper of MSI](#). They are set to 1 (by default) in RSS tool.

Output: The Medical Similarity Index value of the two masks/segmentations.